

Theory and Methodology

Generalized bilinear programming: An application in farm management

Jacqueline M. Bloemhof-Ruwaard^a, Eligius M.T. Hendrix^{b,*}^a Center for Environment and Climate Studies, Agricultural University of Wageningen, P.O. Box 9101, 6700 HB Wageningen, Netherlands^b Department of Mathematics, Agricultural University of Wageningen, Dreijenlaan 4, 6703 HA Wageningen, Netherlands

Received February 1994; revised November 1994

Abstract

In this paper we analyze a model which arises from considering manure and fertilizer aspects in the farm management modelling for cattle husbandry. Modelling these aspects may lead to a multi-extremal problem with bilinear terms. Solution techniques for this problem are discussed and a specific branch and bound procedure is outlined. An example illustrates the multi-extremal structure of the problem and the solution methods mentioned.

Keywords: Agriculture; Environment; Branch and bound; Bilinear programming

1. Introduction

A popular method in farm management and land use planning is linear programming (LP). In this LP context, we formulate the treatment of manure in cattle husbandry. Inputs for a dairy farm are fertilizer, labour, fodder, etc., and saleable outputs are milk, maize or meat. Dairy cows do not only deliver milk, but also produce a large amount of manure that has to be disposed of at some cost. However, this manure (containing useful nutrients) can also be applied for fertilizing the meadow, replacing expensive inorganic fertilizers. Nowadays, the treatment of manure for fertilizing land is an important political issue and a subject of scientific study in the Netherlands.

The government restricts the use of manure for fertilizing the meadow by standards per hectare for each type of land use. The amount of manure to be used for this purpose depends on the land use, which is a decision variable in the model. The resulting model is a *generalized bilinear program*, i.e. a model formulation of which the objective function and the constraint set are linear for one set of decision variables when the other set of variables is fixed and vice versa. We summarize the relevant aspects of this model and discuss some specific characteristics of the new farm management model.

One of the difficulties is the existence of multiple local optima, which makes finding a global optimum an interesting problem. Various solution methods to find global optima are investigated and a specific branch and bound procedure is constructed for the problem described.

* Corresponding author.

The sequel of this paper is organized as follows. Section 2 introduces a mathematical formulation of the problem described above. Additionally an illustrative example is given. Section 3 rewrites the model to a generalized bilinear problem formulation. We compare the nutrient problem with the pooling problem, which is another application of a bilinear program. Also, some theoretical results for the problem are given. Section 4 presents some solution methods. Section 5 closes with some concluding remarks.

2. Formulation of the nutrient problem

In this section treatment of manure is formulated within a farm management model. We call this the *nutrient problem* (NUP) formulation.

Linear farm management models can be used to determine a profit maximizing combination of farm activities that is feasible with respect to a set of fixed farm constraints (Hazell and Norton 1986; Williams, 1990). The variable x_i represents the level of activity i , denoted by the number of hectares. The set I of activities i defines the possibilities in the farm organisation, such as the amount of cattle treated and fed in a particular way. The vector x gives the combination of activities chosen to be the farm plan. Each activity has a profit γ_i (sales minus input costs) and some constraint parameters such as cattle density, amount of labor needed, milk production per hectare, and amount of fodder needed. The constraint parameters build the matrix A , also known as the technology set of the farm model. The set of feasible farm plans is determined by the vector of bounds b , representing, e.g. total land, available labor and allowable milk production (European milk quota).

The traditional farm management model formulation is:

$$\begin{aligned} \max_x \quad & \sum_{i \in I} \gamma_i x_i \\ \text{s.t.} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

The new model covers on one hand decisions on the farm plan with corresponding amounts of

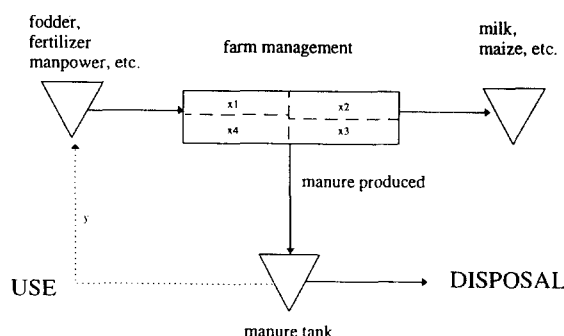


Fig. 1. Scheme of the Nutrient Problem.

manure produced and demands for fertilizing. On the other hand, it covers decisions on the use of manure for fertilizing purposes. Choices in the farm plan determine the possibilities for the application of manure. Fig. 1 outlines the situation.

We assume that all manure of the cattle arrives in one manure tank. The manure in the tank can be used to fertilize the land thus reducing the amount of inorganic fertilizer that has to be bought and applied on the meadow. Furthermore, the re-use of manure saves disposal costs of removal. Disposal costs arise due to manure excess rules in the intensive livestock sector. The variable y_i is introduced to model the decision on the use of the manure for fertilizing the meadow going with farm activity i :

y_i : Fraction of produced manure used on the land with activity i .

No more manure can be applied on the land than has been produced, and thus y_i is restricted by the following constraint:

$$\sum_{i \in I} y_i \leq 1.$$

The cost saving of using the manure produced by activity j , instead of disposing it, is given by the parameter δ_j . This cost saving is higher when the manure contains more of the useful nutrients that the land requires, and also when the prices of the replaced inorganic fertilizer are high. The parameter q_{jk} represents the amount of nutrient k produced in the manure. The index k represents all relevant nutrients, viz. nitrate, phosphate and potassium. In agreement with the approach

of the farm management model, q_{jk} is given in kilogram per hectare. The total amount of nutrient k in the manure is given by

$$\sum_{j \in I} q_{jk} x_j.$$

The government will set standards (parameter d_{ik}) for the utilization of nutrient k per hectare land use of activity i , due to large environmental problems connected with the increased use of nutrients in the intensive livestock sector. The environmental rules can be modelled as

$$y_i \times \sum_{j \in I} q_{jk} x_j \leq d_{ik} x_i, \quad i \in I, \quad k \in K.$$

The left-hand side of these nutrient constraints represents the amount of nutrient k (in kilograms) which is applied on land of activity i by using manure as fertilizer. Notice that by modelling the treatment of manure in this way, the model loses its linearity.

The nutrient problem formulation is:

$$z(\text{NUP}_{x,y}) = \max_{x,y} \sum_{i \in I} \gamma_i x_i + \sum_{i \in I} y_i \sum_{j \in I} \delta_j x_j \quad (1)$$

$$\text{s.t.} \quad Ax \leq b, \quad (2)$$

$$\sum_{i \in I} y_i \leq 1, \quad (3)$$

$$d_{ik} x_i - y_i \sum_{j \in I} q_{jk} x_j \geq 0, \quad i \in I, \quad k \in K, \quad (4)$$

$$x_i, y_i \geq 0, \quad i \in I. \quad (5)$$

This problem can be seen as a bilinear programming problem, which occurs in the literature (see, e.g. Al-Khayyal, 1992). Another application of a bilinear programming problem is the pooling problem, which seems to be closely related to the nutrient problem. Section 3 analyzes these problems in more detail.

To illustrate the formulations, ideas and solution methods in this paper, we use a simple example with two possible farm activities (1 and 2) and environmental rules for three nutrients.

The corresponding $\text{NUP}_{x,y}$ model formulation is:

$$\max_{x,y} \quad 20x_1 + 70x_2 + (y_1 + y_2)(70x_1 + 95x_2) \quad (6)$$

$$\text{s.t.} \quad x_1 + x_2 \leq 10, \quad (7)$$

$$x_1 + 1.5x_2 \leq 12, \quad (8)$$

$$y_1 + y_2 \leq 1, \quad (9)$$

$$y_1(x_1 + 5x_2) \leq x_1, \quad y_2(x_1 + 5x_2) \leq x_2, \quad (10)$$

$$y_1(4x_1 + x_2) \leq 5x_1, \quad y_2(4x_1 + x_2) \leq 4x_2, \quad (11)$$

$$y_1(x_1 + x_2) \leq x_1, \quad y_2(x_1 + x_2) \leq x_2, \quad (12)$$

$$x_1, x_2, y_1, y_2 \geq 0. \quad (13)$$

The classical farm management part consists of two constraints (7) and (8) that handle an upper bound on the total area used and an upper bound on the production of milk. In constraint (9), the fraction of manure applied to the meadow of activities 1 and 2 is less than or equal to 1. The nutrient constraints (10), (11) and (12) represent the environmental rules related to the three nutrients. In this example, constraints (11) and (12) are redundant due to constraint (10). For data instances with this property (the *one-nutrient problems*) specific characteristics can be derived (cf. Sections 3.3 and 3.4).

3. Analysis of the model

In this section, the nutrient model is analyzed in more detail. We discuss the relation with the pooling problem and bilinear problems in general and show the possibility of decomposing the problem. Moreover, we derive the boundary character of the solutions of the nutrient problem.

3.1. Pooling property

The nonlinearity in (1) and (4) is caused by the fact that manure is mixed in a manure tank. The bilinearity in the constraints describing the quality of a product is called the *pooling property*, which occurs in the pooling problem (Foulds et al., 1990). The pooling problem appears frequently in process optimization, especially in refinery planning models. When crudes are supplied to the refinery, they mostly have to be

stored in tanks to be refined later. Due to the limited capacity of the storage tanks, several crudes (with different qualities) will have to be pooled. The clients of the firms ask for end products with a specific quality. The objective is to maximize profit, defined by sales of end products minus the purchase costs of the crudes. The end products arise from a mixture of the crudes, so that the quality of the end products depends on the quality of the crudes in the different tanks. This pooling property causes an inherent nonlinearity in the otherwise linear model. In fact, the pooling model is bilinear. Various methods for solving the pooling problem have been studied, and most of these are based on successive linear programming. Foulds et al. (1990) proposes an algorithm based on a known branch and bound procedure by Al-Khayyal and Falk (1983).

The nutrient problem has the same characteristics as the pooling problem because it is linear whenever either the decision vector x or y is fixed. However, the pooling problem has fixed bounds on the concentrations of the end product, whereas in the nutrient problem the concentrations may vary and the use of the end product is bounded by the input, i.e. the farm plan. One could say that the output (concentrations) is cyclically connected with the input (the farm plan), see Fig. 1. The pooling problem as described in Foulds et al. (1990) has a linear objective function with bilinear balance restrictions whereas the nutrient problem $NUP_{x,y}$ has a bilinear objective function and bilinear constraints. In the next section this type of problem is discussed.

3.2. The nutrient problem as a bilinear problem

The roots of bilinear programming can be found in Nash (1951), who introduced game problems involving two players. Each player must select a mixed strategy from fixed sets of strategies open to each, given knowledge of the payoff based on selected strategies. These problems can be treated by solving a bilinear program.

A bilinear problem is

$$\min_{x,y \in \Omega} f(x, y) = c^T x + x^T A y + d^T y.$$

The objective is a function of two groups of variables. The problem is linear in one group of variables if the other group is fixed, and vice versa, over the feasible region Ω . Bilinear problems are interesting from a research point of view, because of the numerous applied problems that can be formulated as bilinear programs (dynamic Markovian assignment problems, multi-commodity network flow problems, quadratic concave minimization problems).

In the first model formulation, the feasible set Ω for x and y is the cartesian product $X \times Y$, where

$$X := \{x : B_1 x \leq b_1, x \geq 0\},$$

$$Y := \{y : B_2 y \leq b_2, y \geq 0\}.$$

Global optimization algorithms to solve these *traditional bilinear programs* (TBP) have been developed, e.g. by Falk (1973) and Sherali and Shetty (1980). These algorithms guarantee finite convergence for all instances. The key property of the traditional model, used in almost all methods, is that the feasible region is expressed as the cartesian product of two polyhedra. This structure ensures the existence of an extreme-point global solution (Falk, 1973).

Al-Khayyal (1990) relaxed this assumption by considering the feasible set

$$\Omega := \{(x, y) : Cx + Dy \leq b, x \geq 0, y \geq 0\}$$

in the *jointly constrained bilinear program* (JCBP). In this case, interaction in the constraints between the x and y decision variables is allowed.

A further extension of the model is to include bilinear constraints, which results in the *generalized bilinear program* (GBP):

$$z(\text{GBP}) = \min_{x,y \in \Omega} c_0^T x + x^T A_0 y + d_0^T y$$

with

$$\Omega := \{(x, y) : c_p^T x + x^T A_p y + d_p^T y \leq b_p,$$

$$p = 1, \dots, r, Cx + Dy \leq b, x \geq 0, y \geq 0\}.$$

The nutrient problem $NUP_{x,y}$ now becomes a special case of GBP where constraints (2) and (3)

represent the linear parts and constraints (4) the bilinear part. Note that for this special case the jointly constrained part of the GBP reduces to the cartesian product $X \times Y$, where

$$X := \{x : Ax \leq b, x \geq 0\},$$

$$Y := \left\{y : \sum_{i \in I} y_i \leq 1, y \geq 0\right\}.$$

The objective function of a generalized bilinear problem is a *bi-affine* function, that is affine for x and y for fixed y and x respectively. The feasible set Ω is a *bipolyhedron*, i.e. a polyhedron in x and y for fixed y and x respectively. A general way to analyze such a bilinear problem is to fix x and y successively. Section 4.1 describes this method.

3.3. The nutrient problem as a two step problem

The special structure of the nutrient problem makes it possible to analyze the problem from the following point of view. For every x , the optimal y can be calculated. Given this optimal value for y (in terms of x variables), we only have to solve a maximization problem in the x variables. The branch and bound algorithm in Section 4.3 uses this feature.

For convenience, we introduce S as the sum of the y variables, and $S(x)$ as the optimal fraction of the manure produced, applied on the land given farm plan x . Now, the nutrient problem in x variables is:

$$z(\text{NUP}_x) = \max_x (\gamma + S(x) \times \delta)^T x$$

$$\text{s.t. } Ax \leq b, \\ x \geq 0.$$

The objective function is monotonically increasing in S since δ is strictly positive. The objective value corresponding to the maximization problem is denoted as $z(x, S(x))$. In general, the expression $S(x)$ is equivalent to

$$\sum_{i \in I} \min_k \frac{d_{ik} x_i}{\sum_{j \in I} q_{jk} x_j}.$$

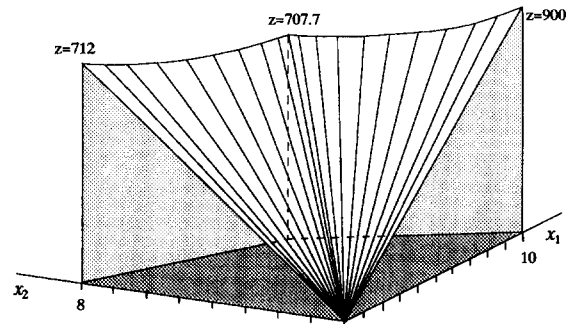


Fig. 2. Graphical representation of (NUP_x) for the example.

If only one nutrient constraint is active, the expression for $S(x)$ is much easier. For the example, the sum $S(x)$ can be expressed as

$$\frac{x_1 + x_2}{x_1 + 5x_2},$$

since (10) is the only active constraint. This expression can be substituted in the objective function. For the resulting nonlinear problem, the first-order Karish–Kuhn–Tucker conditions can be analyzed (see Appendix A). The global maximum is $x^* = (10, 0)^T$ with corresponding optimal value $(1, 0)^T$ for the y variables and an objective value of 900.

Fig. 2 gives the graphical representation of the (NUP_x) model for this example.

The example shows that it is interesting to examine the boundary of the feasible region of the variables $x_1, \dots, x_m$ in particular, since all local optima can be found there. Moreover, Al-Khayyal and Falk (1983) show that jointly constrained bilinear problems always have boundary solutions.

3.4. Boundary solutions

In this subsection, we analyze the existence of boundary solutions for the nutrient problem. The property of boundary solutions can be used explicitly in branch and bound methods as will be outlined in Section 4.3, since this characteristic can reduce the computation time of these algorithms.

We first consider the one-nutrient problem, where one nutrient constraint (say k^*) makes the other nutrient constraints redundant. For this simplification, the optimal value for $y_i(x)$ is

$$\frac{d_{ik} \cdot x_i}{\sum_{j \in I} q_{jk} \cdot x_j}.$$

Therefore, $S(x) := \sum_{i \in I} y_i(x)$ can be written as $d^T x / (q^T x)$, and the objective function (1) can be replaced by

$$\min - \left(\gamma^T x + \frac{x^T \delta d^T x}{q^T x} \right) = \min \frac{x^T C x}{q^T x}$$

with

$$C = -\langle \gamma q^T + \delta d^T \rangle.$$

The feasible region

$$X = \{x : Ax \leq b; x \geq 0\}$$

is compact.

Theorem 1. *If the matrix C has at least one negative eigenvalue, the optimal solution x^* can be found on the boundary of the feasible region X .*

The proof of this theorem is by indirect demonstration and is based on the existence of a direction in which the objective is concave. If x^* is an interior point then it is always possible to obtain a better function value in the neighbourhood of x^* . So, x^* cannot be a minimum point. More details on this can be found in Bloemhof-Ruwaard and Hendrix (1993) or alternatively in Pardalos (1986).

In general, matrix C in the one-nutrient problem has such a negative eigenvalue. As the vectors d , q and δ are positive, it is sufficient to have at least one positive element of the vector γ if the corresponding q -element has a strictly positive value. So, at least one farm plan should have a positive contribution, which is not a severe assumption.

We can generalize this result to the general nutrient problem. First, Al-Khayyal and Falk (1983) show that the solution must be on the boundary of the $X \times Y$ space.

Theorem 2 (Al-Khayyal and Falk, 1983). *If an optimal solution exists to the GBP problem, at least one optimal solution will be on the boundary of the bipolyhedral feasible set.*

This theorem states that some optimal solution (x^*, y^*) of $\text{NUP}_{x,y}$ is found on the boundary of Ω . This solution is not necessarily an extreme point (Al-Khayyal, 1990).

For the nutrient problem, Theorem 3 shows that (x^*, y^*) is on the boundary of X .

Theorem 3. *Given problem $\text{NUP}_{x,y}$ defined by (1)–(5), and*

$$X := \{x \in \mathbb{R}^n : Ax \leq b; x \geq 0\}$$

is a compact set. Then the solution (x^, y^*) of $\text{NUP}_{x,y}$ is found on the boundary of X .*

Proof. Given the optimal y^* for a GBP problem, (x^*, y^*) is a solution of a linear programming problem in x only. Let $\text{NUP}_{x|y^*}$ denote the nutrient problem (NUP) given a fixed value y^* for y . Constraints (4) define a cone

$$\mathcal{X} := \left\{ x \in \mathbb{R}^n : (d_{ik} e_i - y_i^* q_k)^T x \geq 0, \right. \\ \left. i \in I, k \in K \right\},$$

with e_i the i -th unit vector. The solution (x^*, y^*) can be found at a vertex of the feasible set of $\text{NUP}_{x|y^*}$ consisting of $X \cap \mathcal{X}$. Apart from the origin, $\mathcal{X}$ does not contain a vertex. If x^* is not the origin, then it is not a vertex of $\mathcal{X}$, so the solution is found on the boundary of X . $\square$

4. Solution approaches

In this section, classical techniques for solving the nutrient problem are presented. A new branch and bound method is outlined and illustrated.

4.1. Successive linear programming approach

One promising heuristic to solve a bilinear programming problem is *successive linear programming*. Starting with any $x^0 \in X$, find y^0 that optimizes the objective function over Y , then find

x^1 that optimizes the objective function over X , and so on until the objective does not improve between two successive optimizations.

If the feasible set Ω is a cartesian product $X \times Y$, with X and Y convex sets and the objective function is differentiable and convex over $X \times Y$, then any limit point to this alternating procedure globally solves the problem (Wendell and Hurter, 1976). If the objective function is allowed to be biconvex, every limit point is a Karish–Kuhn–Tucker point (Konno, 1976), but necessary conditions are needed for a limit point to be a local solution to the biconvex problem. However, the GBP problem does not have these characteristics. Therefore, the approach by successive linear programming can be very disappointing, as is illustrated by the nutrient problem.

First note the following observations for the nutrient problem (NUP):

- For any fixed $y^t \in Y$ in step t , the LP-problem in the variables x is equivalent to:

$$z(\text{NUP}_{x|y^t}) = \max_x (\gamma + S^t \delta)^T x \quad (14)$$

$$\text{s.t. } d_{ik} x_i \leq (q_k^T x) y_i^t, \quad i \in I, \quad k \in K, \quad (15)$$

$$Ax \leq b, \quad (16)$$

$$x \geq 0. \quad (17)$$

- For any fixed $x^t \in X$ in step t , the LP-problem in the variables y is equivalent to:

$$z(\text{NUP}_{y|x^t}) = \max_y \left(S = \sum_{i \in I} y_i \right) \quad (18)$$

$$\text{s.t. } y_i (q_k^T x^t) \leq d_{ik} x_i^t, \quad i \in I, \quad k \in K, \quad (19)$$

$$S \leq 1, \quad (20)$$

$$y \geq 0. \quad (21)$$

- A feasible solution for the y -variables can be found for any given $x \in X$. The reverse statement does not necessarily hold.

The successive linear programming procedure for the $\text{NUP}_{x,y}$ problem is described as follows:

Successive Linear Programming Algorithm

Step 1. Start with any feasible y^0 , $t = 1$.

Step 2. Given y^{t-1} (with corresponding value for S^{t-1}), find x^t that minimizes $z(\text{NUP})_{x|y^{t-1}}$ over the feasible region for x .

Step 3. Given x^t find y^t that minimizes $z(\text{NUP})_{y|x^t}$ over the feasible region for y .

Step 4. If $|y^t - y^{t-1}| \leq \epsilon$ then STOP, else $t := t + 1$ and goto Step 2.

For the example, the successive linear programming algorithm stops after at most two iterations. The LP solution $x^* = x^1$ is a point on the boundary of X . Only for the starting vector $y^0 = (1, 0)^T$, the successive linear programming method finds the global maximum $x^* = (10, 0)^T$. This example illustrates that the successive linear programming approach may be very unsuccessful for the nutrient problem.

4.2. Nonlinear programming approach: Local search

Perhaps the most common method to solve problems like the nutrient problem is to apply standard nonlinear programming software. As illustrated in Fig. 2, NUP may contain many local optima. One could try to discover the global optimum by starting a nonlinear programming local search from various starting points. An overview of global optimization methods, based on local searches, can be found in Törn and Zilinskas (1989). One of the known packages for solving constrained nonlinear programming is GAMS/MINOS. Implementation of the example in this package (MINOS 5.3) gives the different local optima for various starting points (Table 1).

Table 1
Solutions local search

Starting point				Local optimum				Objective
x_1	x_2	y_1	y_2	x_1	x_2	y_1	y_2	
0.0	0.0	0.0	0.0	10	0	1.00	0.00	900.0
3.0	3.0	0.2	0.2	6	4	0.23	0.15	707.7
3.0	0.0	0.0	0.0	0	8	0.00	0.20	712.0

Starting point (0, 0, 0.2, 0.2) did not converge to a local optimum, but stayed at the starting point. In the approach of using various starting points, at least one of the points should be situated in the region of attraction of the global optimum, which is hard to verify. Finding the global optimum requires use of the special structure of the problem. This can be done in a branch and bound environment.

4.3. Branch and bound approach

Many global optimization problems can be tackled by means of a branch and bound algorithm. These problems belong to a special class, such as concave minimization, DC-programming, quadratic programming or bilinear programming. Often linear programs are used as subproblems that are solved in a partition scheme for finding global solutions. A general framework for branch and bound in global optimization can be found in Horst and Tuy (1990).

4.3.1. Branch and bound algorithms for bilinear programs

Falk (1973) develops the first branch and bound procedure for the traditional bilinear program. It guarantees convergence to an exact solution in a finite number of steps. Bounds are obtained by solving linear programming relaxations and branching is accomplished by holding individual variables out of the basis for each branch when using the simplex method to solve the subproblems. Al-Khayyal and Falk (1983) develop a branch and bound algorithm for the JCBP problem. The algorithm branches into four nodes based on a partition of the parent node's rectangular set

$$C = \{(x, y) : l_x \leq x \leq u_x, l_y \leq y \leq u_y\}$$

and bounding is achieved by minimizing the convex envelope of the objective function over the subset of the feasible region, $\Omega \cap C^p$, for the p -th partition set considered. The same can be done for the GBP problem (Al-Khayyal, 1992). In the next section, we will not make use of this general bilinear approach, but we will develop a specific

branch and bound method for the nutrient problem.

4.3.2. A Branch and bound algorithm for the nutrient problem

We will now consider a special branch and bound algorithm for the nutrient problem. In contrast with Al-Khayyal (1992), partition sets are defined only in the X -space, not in the $X \times Y$ space. Furthermore, bounds are not based on convex envelopes, but on the monotonicity of $z(x, S(x))$, described in the decomposition approach of Section 3.3.

The rectangular partition set $C^p : [l^p, u^p]$ is defined as the p -th block created in the branching tree (l and u have the same dimension as x). In each iteration of the branch and bound algorithm we need to calculate the maximum of $S(x)$ over all $x \in (C^p \cap X)$. This value is not easy to find, so we try to find a good upperbound σ^p for $S(x)$ that is easy to obtain.

First the determination of σ^p is discussed, then the branch and bound framework is outlined.

The upperbound derivation of the maximum value for S in (the feasible part of) each box C^p is as follows:

$$\begin{aligned} \max_{x \in C^p \cap X} S(x) &= \max_{x \in C^p \cap X} \left\{ \sum_i \min_k \frac{d_{ik} x_i}{q_k^T x} \right\} \\ &\leq \max_{x \in C^p} \left\{ \sum_i \min_k \frac{d_{ik} x_i}{q_k^T x} \right\} \\ &\leq \max_{x \in C^p} \min_k \left\{ \frac{d_k^T x}{q_k^T x} \right\} \leq \min_k \max_{x \in C^p} \left\{ \frac{d_k^T x}{q_k^T x} \right\} \end{aligned}$$

So we choose

$$\sigma^p := \min \left\{ 1, \min_k \max_{x \in C^p} \left\{ \frac{d_k^T x}{q_k^T x} \right\} \right\}.$$

First, a fractional programming problem has to be solved for each k . Appendix B.1 describes a solution method for such a problem, *the threshold algorithm*. It is easy to determine σ^p once the optimal solution for each fractional programming problem is found.

The formal branch and bound description is:

Branch and bound Algorithm

Step 0. Give tolerance ε .

Step 1. The list P consists only one box $C^1 = [l^1, u^1] \supset X$

Calculate σ^1 . Initiate z_U as

$$z_u^1 := \max_{x \in X} z(x, \sigma^1).$$

The value of x corresponding to this maximum is denoted by x^1 .

For this x^1 initiate z_L as

$$z_l^1 := z(x^1, S^1) \text{ and } x_{\text{best}} := x^1.$$

Step 2. Divide the box under consideration over the longest edge into two boxes of equal volume and for both boxes C^p goto Step 3.

Step 3. First perform the interior check:

If the box is interior, stop analyzing the box else determine σ^p .

Determine

$$z_u^p := \max_{x \in C^p \cap X} z(x, \sigma^p),$$

and the corresponding solution x^p .

If the box does not contain feasible solutions, stop analyzing the box.

If $z_u^p > z_L$, then determine S^p and

$$z_l^p := z(x^p, S^p).$$

If $z_l^p > z_L$, then

$$z_L := z_l^p \text{ and } x_{\text{best}} := x^p.$$

Put the box C^p in the list of boxes.

Step 4. Delete all boxes in the list with $z_u^p < z_L$.

Delete all boxes which are too small.

If the list is empty, then STOP.

Step 5. Search for the box C^p in the list P with the highest upper bound:

$z_U := z_u^p$, this is the new box we consider.

If $|z_U - z_L| < \varepsilon$, then STOP,

else goto Step 2.

The algorithm starts with the smallest rectangular box C^1 that includes

$$X := \{x : Ax \leq b; x \geq 0\}$$

as a whole. Note that the interior of C^1 is strictly positive. In Step 1, the value of σ^1 is obtained using the threshold algorithm of Appendix B.1. A global upper bound to $z(\text{NUP}_{x,y})$, z_U , is found by maximizing $z(x, \sigma^1)$ over X . For this solution x^1 , we calculate the optimal y and the corresponding

$S(x^1)$ solving the linear programming problem $\text{NUP}_{y|x^1}$ and the first lower bound $z_L = z(x^1, S(x^1))$ is found.

At each iteration, there is a list P of boxes C^p . One of them is split by dividing the box into two parts of equal volume over the longest edge (bisection). This creates two new boxes (Step 2).

For each of these boxes C^p it is checked whether the box is totally interior with respect to X . If it is, it is discarded and we proceed with another box. Appendix B.2 describes a suitable check.

If the box is not interior, we continue with Step 3. A new upper bound $z_u^p := z(x^p, \sigma^p)$ is calculated, based on the upperbound σ^p obtained from the threshold algorithm described in Appendix B.1. A feasible solution (x^p, S^p) is obtained by solving the simple LP problem $\text{NUP}_{y|x^p}$, which results in a new lower bound $z_l^p := z(x^p, S^p)$. If this lower bound is better than z_L , then $z_L := z_l^p$ and the solution x^p is saved.

In Step 4 the list of boxes is checked: If the upper bound z_u^p of some box C^p is below the new lower bound z_L , there is no use to analyze this box C^p any further, so this box can be neglected.

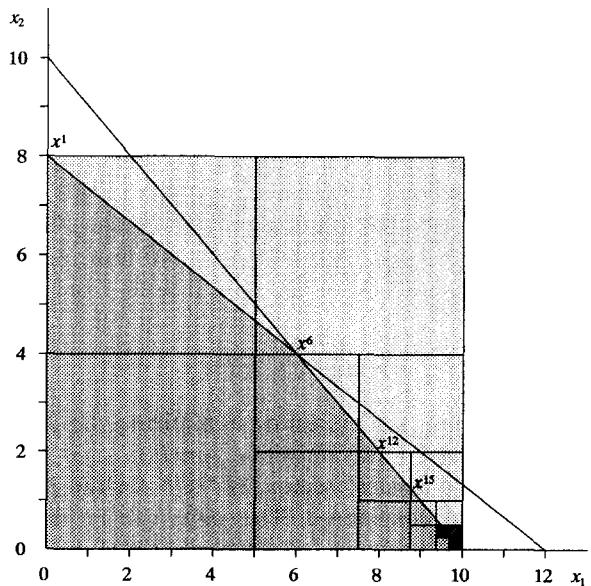


Fig. 3. Partition after 15 iterations.

We select a new box for further division by means of the highest upper bound z_u^p , which is the new z_U (Step 5). During the branch and bound process, the gap between upper and lower-bound, $|z_U - z_L|$, is decreasing. The algorithm stops if this gap is less than a preset tolerance ϵ , or if the list of boxes is empty. In order to guarantee convergence of the overestimate σ^p to the maximum of $S(x)$ and to fasten the algorithm, boxes that are too small are discarded.

Application of the branch and bound procedure to the example leads to the following result. After 15 iterations, the point $x_{\text{best}} = (9.75, 0.25)$ gives the best lower bound $z_L = 855$. The best upper bound is $z_U = 919$, so these bounds enclose the optimum value $z^* = 900$ at $(10, 0)$. Only three small partition sets in the neighbourhood of $x^* = (10, 0)$ remain on the list (see Fig. 3).

5. Conclusion

Including the return flow of manure in a farm management model gives a special structured problem, the nutrient problem. This problem appears to be a special form of a generalized bilinear programming (GBP) problem and is closely related to the pooling problem. This type of problem often has multiple local optima. For the nutrient problem, at least one optimal solution will be found at the boundary of the total feasible set, and also at the boundary of the feasible set of farm plans.

We analyzed the use of global optimization methods for the nutrient problem and illustrated their behaviour with an example. The successive linear programming approach for bilinear problems, successful on convex problems, is not satisfactory. The example shows that a global optimum is only found if it is a starting point for the procedure. This result holds in general and does not depend on a particular data set. The use of nonlinear programming codes can result in different local optima for different starting points. These codes can be applied as local optimization procedures in a multi-start method in an attempt to obtain the global optimum. The global optimum can always be obtained by a branch and

bound procedure. A general procedure for GBP problems leads to subsets in the common feasible area $(X \times Y)$ of the decision variables.

Due to a decomposition of the nutrient problem, the subsets used for branching can be defined in the X -space of farm plans. In this specific branch and bound procedure, upper estimates are obtained by an algorithm for fractional programming on a hyperrectangle. The boundary characteristic helps to fasten the outlined branch and bound procedure.

Appendix A. Karish–Kuhn–Tucker conditions

The constrained nonlinear programming (NLP) problem involves maximization of a smooth nonlinear function subject to smooth inequality constraints on a finite set of continuous variables.

The problem is:

$$\begin{aligned} z(\text{NLP}) = \max \quad & f(x_1, x_2, \dots, x_n) \\ \text{s.t.} \quad & g_i(x_1, x_2, \dots, x_n) \leq b_i, \quad i = 1, \dots, m. \end{aligned} \quad (\text{A.1})$$

Under certain regularity conditions (see Bazaraa and Shetty, 1979, p. 137) the following theorem holds:

Theorem A.1. *If $\bar{x} = (\bar{x}_1, \dots, \bar{x}_n)$ is an optimal solution to NLP, then $\bar{x} = (\bar{x}_1, \dots, \bar{x}_n)$ must satisfy the m constraints in (A.1), and multipliers $\bar{\lambda}_1, \dots, \bar{\lambda}_m$ exist, satisfying*

$$\frac{\partial f(\bar{x})}{\partial x_j} - \sum_{i=1}^m \bar{\lambda}_i \frac{\partial g_i(\bar{x})}{\partial x_j} = 0, \quad j = 1, \dots, n, \quad (\text{A.2})$$

$$\bar{\lambda}_i [b_i - g_i(\bar{x})] = 0, \quad i = 1, \dots, m, \quad (\text{A.3})$$

$$\bar{\lambda}_i \geq 0, \quad i = 1, \dots, m. \quad (\text{A.4})$$

These conditions are known as the Karish–Kuhn–Tucker conditions (KKT-conditions).

For our example the NLP problem is:

$$\begin{aligned} \max \quad & f(x) = 20x_1 + 70x_2 \\ & + \left(\frac{x_1 + x_2}{x_1 + 5x_2} \right) \times (70x_1 + 95x_2) \\ \text{s.t.} \quad & x_1 + 1.5x_2 \leq 12, \\ & x_1 + x_2 \leq 10, \end{aligned}$$

$$-x_1 \leq 0,$$

$$-x_2 \leq 0.$$

If $(x_1, x_2) = (0, 0)$, then $f(0, 0) := 0$.

The corresponding KKT-conditions are:

1. $(x_1, x_2) \in X$, where

$$X = \{x_1 \geq 0, x_2 \geq 0 \mid x_1 + x_2 \leq 10; \\ x_1 + 1.5x_2 \leq 12\}.$$

$$2. \frac{\partial f(x)}{\partial x_j} - \sum_{i=1}^4 \lambda_i \frac{\partial g_i(x)}{\partial x_j} = 0 \\ \text{for } j = 1, 2.$$

$$3. \lambda_1 = 0 \text{ or } x_1 + 1.5x_2 = 12, \\ \lambda_2 = 0 \text{ or } x_1 + x_2 = 10, \\ \lambda_3 = 0 \text{ or } x_1 = 0, \\ \lambda_4 = 0 \text{ or } x_2 = 0.$$

$$4. \lambda_1 \geq 0, \lambda_2 \geq 0, \lambda_3 \geq 0, \lambda_4 \geq 0.$$

After some calculations we get

$$\frac{\partial f(x)}{\partial x_1} - \sum_{i=1}^4 \lambda_i \frac{\partial g_i(x)}{\partial x_1} = 0 \\ \Leftrightarrow \frac{90x_1^2 + 900x_1x_2 + 1230x_2^2}{x_1^2 + 10x_1x_2 + 25x_2^2}$$

$$- \lambda_1 - \lambda_2 + \lambda_3 = 0,$$

$$\frac{\partial f(x)}{\partial x_2} - \sum_{i=1}^4 \lambda_i \frac{\partial g_i(x)}{\partial x_2} = 0 \\ \Leftrightarrow \frac{-115x_1^2 + 890x_1x_2 + 2225x_2^2}{x_1^2 + 10x_1x_2 + 25x_2^2} \\ - 1.5\lambda_1 - \lambda_2 + \lambda_4 = 0.$$

The solutions to these equations give the KKT-points listed in Table A.1 (KKT-points not in the feasible region of (x_1, x_2) or $(\lambda_1, \dots, \lambda_4)$ are eliminated).

After some examination of the gradients of f in all directions we can conclude:

• KKT-point 1 is a local maximum in all feasible directions.

• KKT-point 2 is a local minimum over the boundary but is a local maximum over the line

$$x_2 = \frac{5.6649}{3.5027} \times x_1$$

in the interior region.

• KKT-point 3 is a local maximum in all feasible directions.

• KKT-point 4 is a local minimum over the boundary but is a local maximum over the line

$$x_2 = \frac{3.1458}{6.8543} \times x_1$$

in the interior region.

• The global maximum is obtained in $(10, 0)$.

Appendix B. Algorithms

B.1. Threshold algorithm

For each nutrient k the fractional programming problem

$$\max_{x \in C^p} \phi_k(x) := \frac{d_k^T x}{q_k^T x}$$

has to be solved, where the demand and supply vectors (d and q) are strictly positive.

A well-known approach to solve fractional programming problems (FP)

$$\max_{x \in X} f(x)/g(x)$$

is to consider the global optimization problem (GP):

$$\max_{x \in X} \{f(x) - \lambda g(x)\}$$

where $\lambda \in \mathbb{R}$ (Pardalos and Phillips, 1991). The

Table A.1

r	x_1 -value	x_2 -value	z -value	λ_1	λ_2	λ_3	λ_4
1	0	8	712	59.3333	0	10.1333	0
2	3.5027	5.6649	692.2338	57.6862	0	0	0
3	6	4	707.6923	24.5562	41.3018	0	0
4	6.8542	3.1458	702.0794	0	70.2079	0	0
5	10	0	900	0	90	0	205

fundamental result which relates the GP problem to the FP problem is

Theorem B.1 (Dinkelbach, 1967). x^* solves the fractional programming problem FP if and only if x^* solves the global optimization problem GP with constant $\lambda^* = f(x^*)/g(x^*)$.

For our fractional programming problems, according to this theorem,

$$\lambda_k^* = \frac{d_k^T x^*}{q_k^T x^*}.$$

For this constant λ_k^* , the global optimization problem is the linear programming problem

$$\max_{x \in C^p} \sum_{i \in I} (d_{ik} - \lambda_k^* q_{ik}) x_i.$$

If $d_{ik} - \lambda_k^* q_{ik} < 0$, then it is optimal to decrease x_i as far as possible ($x^* = l_i^p$), and if $d_{ik} - \lambda_k^* q_{ik} \geq 0$, then it is optimal to increase x_i as far as possible ($x_i^* = u_i^p$).

So

$$x_i^* = \begin{cases} l_i^p, & \text{if } d_{ik}/q_{ik} < \lambda_k^*, \\ u_i^p & \text{if } d_{ik}/q_{ik} \geq \lambda_k^*. \end{cases}$$

For each k , λ_k^* can be seen as the *threshold* value, which determines in which corner of the box C^p the value x^* is obtained. An algorithm to find this x^* is described below:

Threshold Algorithm

Step 1. Fix all x_i at l_i^p and calculate the objective value ϕ_k .

Initiate ϕ_k^* as ϕ_k .

Step 2. Renumber all x_i , such that

$$\frac{d_{1k}}{q_{1k}} \leq \frac{d_{2k}}{q_{2k}} \leq \dots \leq \frac{d_{mk}}{q_{mk}}.$$

Pick up the first element of the sorted vector x .

Step 3. Put the chosen element x_i on its upper bound u_i^p and calculate the new ϕ_k .

If ϕ_k exceeds ϕ_k^* , then goto Step 4

else put the chosen element back to its lower bound l_i^p and STOP:

ϕ_k^* is the optimum.

Step 4. Replace ϕ_k^* by the new ϕ_k and take the next element of the x vector, goto Step 3.

B.2. Interior check algorithm

To check whether a box $C^p = [l^p, u^p]$ is interior with respect to the polyhedron X , we write X as a set of linear inequalities $\{Ux \leq v\}$, where v_i ($i = 1, \dots, n$) is the right-hand-side of the i -th linear inequality.

Interior check Algorithm

Step 0. $i = 1$.

Step 1. Calculate the vector

$$w = (w_1, \dots, w_j, \dots, w_m)^T$$

where

$$w_j = \begin{cases} l_j^p & \text{if } u_{ij} < 0, \\ u_j^p & \text{if } u_{ij} \geq 0. \end{cases}$$

Step 2. If $u_i^T w \geq v_i$, then STOP; box is not interior

else $i := i + 1$.

Step 3. If $i = n + 1$, then STOP; box is interior else goto Step 1.

Acknowledgements

The authors thank Luk Van Wassenhove, Leen Hordijk and Hans van Maaren for reading and commenting on earlier versions of the paper. The useful comments provided by two anonymous referees have improved the quality of presentation of the paper.

References

- Al-Khayyal, F.A. (1990), "Jointly constrained bilinear programs and related problems: An overview", *Computers & Mathematics with Applications* 19, 53–62.
- Al-Khayyal, F.A. (1992), "Generalized bilinear programming: Part I. Models, applications and linear programming relaxation", *European Journal of Operational Research* 60, 306–314.
- Al-Khayyal, F.A., and Falk, J.E. (1983), "Jointly constrained biconvex programming", *Mathematics of Operations Research* 8, 273–286.
- Bazaraa, M., and Shetty, C. (1979), *Nonlinear Programming: Theory & Algorithms*, Wiley, New York.
- Bloemhof-Ruwaard, J.M., and Hendrix, E.M.T. (1993), "Generalized bilinear programming: An application in farm

- management", Technical Paper 93.05, Center for Environment and Climate Studies, Agricultural University of Wageningen, Netherlands.
- Dinkelbach, W. (1967), "On nonlinear fractional programming", *Management Science* 13, 492–498.
- Falk, J.E. (1973), "A linear max-min problem", *Mathematical Programming* 5, 169–188.
- Foulds, L.R., Haugland, D., and Jörnsten, K. (1990), "A bilinear approach to the pooling problem", CMI Working Paper 90/03, Bergen, Norway.
- Hazell, P.B.R., and Norton, R.D. (1986), *Mathematical Programming for Economic Analysis in Agriculture*, Macmillan, New York.
- Horst, R., and Tuy, H. (1990), *Global Optimization*, Springer-Verlag, Berlin.
- Konno, H. (1976). "A cutting plane algorithm for solving bilinear programs", *Mathematical Programming* 11, 14–27.
- Nash, J.F. (1951), "Noncooperative games", *Annals of Mathematics* 54, 286–295.
- Pardalos, P.M. (1986), "Global optimization algorithms for indefinite quadratic problems", Report CS-86-29, Department of Computer Science, Pennsylvania State University, University Park, PA 16802.
- Pardalos, P.M., and Phillips, A.T. (1991), "Global optimization of fractional programs", *Journal of Global Optimization* 1, 173–182.
- Sherali, H.D., and Shetty, C.M. (1980), "A finite convergent algorithm for bilinear programming problems using polar cuts and disjunctive face cuts", *Mathematical Programming* 19, 14–31.
- Törn, A., and Žilinskas, A. (1989), *Global Optimization*, Lecture Notes in Computer Science 350, Springer-Verlag, Berlin.
- Wendell, R.E., and Hurter, A.P. (1976), "Minimization of a non-separable objective function subject to disjoint constraints", *Operations Research* 24, 643–657.
- Williams, H.P. (1990), *Model Building in Mathematical Programming*, Wiley, New York.